

Project Based Learning (PjBL)

Judul :

**Transformasi Konsep Dasar Grafika Komputer dalam
Pengembangan Game Arcade 'Snake' Berbasis Web
Menggunakan Metode Game Development Life Cycle (GDLC)**

Dosen Pengampu:

Suharsono, S.Kom., M.Kom.

**Mata Kuliah : Grafika Komputer
Semester/Kelas : IV/ 4A dan 4B
Pertemuan : 9 – 10**

**Program Studi D3 Teknik Informatika
Jurusan Teknik Elektro
Politeknik Negeri Pontianak
2026**

1. Perbedaan PBL dan PjBL

Tabel Perbedaan PBL dan PjBL

Fitur	Problem Based Learning (PBL)	Project Based Learning (PjBL)
Fokus Utama	Memecahkan masalah/studi kasus	Menciptakan produk/karya
Hasil Akhir	Solusi, rekomendasi, laporan, presentasi	Produk fisik, digital, pameran, karya seni
Durasi	Relatif singkat (beberapa pertemuan)	Jangka panjang (minggu/bulan)
Aktivitas	Investigasi, analisis, riset mandiri	Perencanaan, perancangan, pembuatan proyek
Karakteristik	Menekankan proses berpikir kritis	Menekankan kolaborasi & kreativitas

2. Penjelasan Judul PjBL

1. Aspek Konseptual: Transformasi Konsep Dasar Grafika Komputer

Komponen ini merujuk pada penerapan tiga pilar utama dalam produksi citra digital yang diproses dari deskripsi menjadi gambar visual.

a. Pemodelan (*Modeling*)

Merupakan upaya untuk menggambarkan objek nyata (ular dan makanan) ke dalam bentuk karakteristik geometris matematis, seperti model kerangka (*wireframe*).

b. Rendering

Proses memberikan nuansa realistis pada model melalui pemberian warna, pengaturan cahaya, dan tekstur agar objek menyerupai kondisi sebenarnya di layar.

c. Animasi

Teknik memberikan efek gerakan (*motion*) pada objek grafis yang mengikuti kaidah perpindahan posisi secara sistematis seiring waktu.

2. Aspek Implementasi Geometri: Game Arcade 'Snake'

Objek utama penelitian ini diwujudkan melalui penerapan algoritma pembentukan objek dasar dan sistem koordinat.

a. Output Primitif

Ular dan makanan dimodelkan menggunakan bentuk geometri dasar paling sederhana, yaitu **persegi (*rectangle*)** atau kumpulan piksel.

b. **Sistem Koordinat Perangkat**

Penempatan setiap segmen tubuh ular dihitung berdasarkan koordinat layar monitor, di mana titik **(0,0)** biasanya terletak di pojok kiri atas.

c. **Logika Dinamis**

Implementasi mencakup *motion dynamic* (objek bergerak) dan *update dynamic* (perubahan sifat objek seperti penambahan panjang ular setelah makan).

d. **Deteksi Tabrakan (*Collision Detection*)**

Analisis matematis untuk mendeteksi perpotongan koordinat antara kepala ular dengan objek lain atau batas jendela (*clipping window*).

3. Aspek Teknologi: Berbasis Web

Penelitian ini memfokuskan pada penggunaan platform peramban modern sebagai lingkungan eksekusi grafis.

a. **HTML5 Canvas API**

Menyediakan elemen `<canvas>` sebagai area gambar (*drawing area*) yang dapat dimanipulasi melalui kode untuk menampilkan grafis 2D secara interaktif.

b. **JavaScript**

Bertindak sebagai pengontrol logika permainan dan melakukan pemanggilan fungsi grafis melalui konteks 2D atau **WebGL** untuk rendering tanpa memerlukan *plug-in* tambahan.

c. **Manipulasi Piksel**

Penggunaan metode seperti `getImageData` dan `putImageData` untuk memodifikasi nilai warna (RGBA) langsung pada memori kanvas.

4. Aspek Metodologis: Game Development Life Cycle (GDLC)

Ini merupakan kerangka kerja sistematis dalam melakukan penelitian pengembangan (*research and development*).

a. **Pra-produksi**

Tahap perancangan struktur data objek, seperti representasi ular dalam bentuk **kumpulan (*array*) koordinat titik** [sesuai riwayat percakapan].

b. **Produksi**

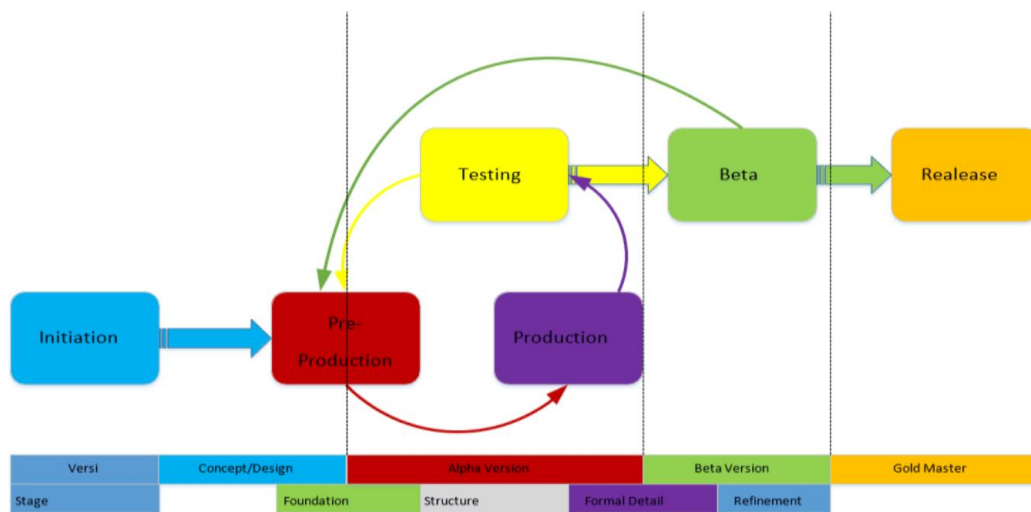
Proses pengodean menggunakan *graphics pipeline* untuk mengubah model digital menjadi gambar piksel di layar.

c. **Pengujian (*Testing*)**

Evaluasi kualitas visual untuk meminimalkan distorsi informasi seperti efek tangga ("**jaggies**") melalui teknik *antialiasing*.

Tahapan Pengembangan (Sesuai Siklus GDLC)

Penggunaan Game Development LifeCycle (GDLC) Merupakan metodologi untuk pengembangan aplikasi berbasis *game* dengan pendekatan iteratif yang terdiri dari 6 *fase* / langkah pengembangan, dimulai pada fase Initialitations atau pembentukan konsep *game*, pra-produksi (*Pre-Production*), produksi *game* (*Production*), ujicoba (*Testing*), *Beta Realease* dan Rilis Aplikasi (*Realease*) [6]. Berikut tahapan GDLC dijelaskan pada Gambar 1.



Gambar 1. *Fase Metode GDLC*

1. Tahap Inisiasi (*Initiation*)

Pada tahap awal ini, pengembang menentukan visi dan aturan dasar permainan.

a. Penjelasan Konsep

Game Snake adalah permainan bertema arkade di mana pemain mengendalikan sebuah objek garis yang terus memanjang (ular). Tujuannya adalah mengumpulkan poin sebanyak-banyaknya dengan memakan objek tertentu di layar. Visi kreatifnya yaitu membuat permainan yang menggunakan representasi objek nyata ke dalam karakteristik geometris yang dapat dimanipulasi secara numeris.

b. Mekanisme Utama:

- 1) Ular bergerak secara konstan ke arah atas, bawah, kanan, dan kiri.
- 2) Ular bergerak secara dinamis (*motion dynamic*) untuk memakan objek di area canvas.

- 3) Ular bertambah panjang setiap kali ular memakan “makanan”.
- 4) Makanan (food) muncul ketika makanan telah dimakan oleh snake.
- 5) Skor akan meningkat (+1 setiap makan) setiap kali ular memakan “makanan”.
- 6) Pemain hanya bisa mengubah arah (belok), bukan menghentikan ular.
- 7) Menggunakan event untuk mengendalikan game (keyboard & mouse).
- 8) Ular dapat menembus dinding dan muncul di dinding berikutnya.
- 9) Ular akan mati/berhenti ketika kepala memakan bagian tubuhnya sendiri.
- 10) Permainan berakhir (*game over*) jika ular mati.

2. Tahap Pra-Produksi (*Pre-production*)

Tahap ini merancang fondasi teknis dan struktur data yang akan digunakan.

a. Sistem Koordinat

Menentukan area permainan menggunakan sistem koordinat layar (perangkat). Sesuai standar sistem grafis digital (seperti GDI+ atau HTML5 Canvas), titik **(0,0) terletak di pojok kiri atas**, dengan nilai x positif ke kanan dan y positif ke bawah.

b. Struktur Data Objek

Mahasiswa harus merancang representasi ular sebagai sebuah **kumpulan (array) dari koordinat titik** dua dimensi. Setiap elemen dalam *array* menyimpan posisi (x,y) dari satu segmen tubuh ular, di mana indeks pertama adalah kepala. Setiap titik diidentifikasi dengan pasangan bilangan bulat yang menunjukkan posisi kolom dan baris piksel. Pemodelan geometri menggunakan *output primitif* berupa persegi (*rectangle*) atau kotak sebagai unit pembentuk tubuh ular dan makanan.

c. Pemilihan Teknologi

Tenologi yang digunakan dalam pengembangan game ini adalah sebagai berikut.

1. **JavaScript** berfungsi sebagai bahasa pemrograman untuk membuat website interaktif dan dinamis. Sedangkan Node.js adalah runtime environment yang memungkinkan kode JavaScript dieksekusi di luar

browser, sehingga berfungsi sebagai pengendali sisi backend (server), pembuatan API, dan pengembangan aplikasi web berskala besar.

2. **HTML5** Canvas adalah elemen antarmuka yang berfungsi sebagai wadah untuk menggambar grafik, visual, dan animasi secara dinamis pada halaman web. Elemen ini menggunakan JavaScript untuk melakukan proses rendering (menggambar) objek seperti bentuk 2D/3D, teks, memanipulasi gambar, hingga membuat game interaktif langsung di peramban (browser) tanpa memerlukan *plug-in* tambahan
3. **Visual Studio Code** sebagai teks editor dalam pengembangan Game. VS Code berfungsi untuk menulis, mengedit, dan men-debug kode sumber aplikasi. Ia mendukung berbagai bahasa pemrograman (JavaScript, Python, PHP, C++, dll) di Windows, macOS, dan Linux dengan fitur utama IntelliSense, manajemen Git terintegrasi, dan ekstensi.
4. **Live Server** di VS Code adalah ekstensi yang berfungsi meluncurkan local development server untuk menampilkan perubahan kode HTML, CSS, dan JavaScript secara real-time di browser. Fitur utamanya adalah memuat ulang (reload) halaman otomatis saat file disimpan, mempercepat proses pengembangan tanpa perlu refresh manual.
5. **Code Runner** adalah menjalankan kode program secara instan untuk berbagai bahasa pemrograman (seperti Python, C, C++, Java, JS) langsung di dalam editor (misalnya VS Code) tanpa konfigurasi rumit. Alat ini memudahkan pengujian cuplikan kode (code snippet), debugging, dan eksekusi skrip, meningkatkan produktivitas developer.

3. Tahap Produksi (*Production*)

Tahap implementasi menggunakan konsep inti grafika komputer.

a. Drawing Area (Canvas)

Membuat area gambar (*drawing area*) yang memiliki properti lebar, tinggi, dan kedalaman warna. Mahasiswa dapat menggunakan API seperti HTML5 Canvas untuk manipulasi piksel secara langsung.

b. Implementasi Animasi

Menggunakan fungsi perulangan untuk memperbarui koordinat segmen tubuh ular secara berkala, menciptakan efek gerakan yang halus.

c. **Transformasi Geometri:**

1. **Translasi**

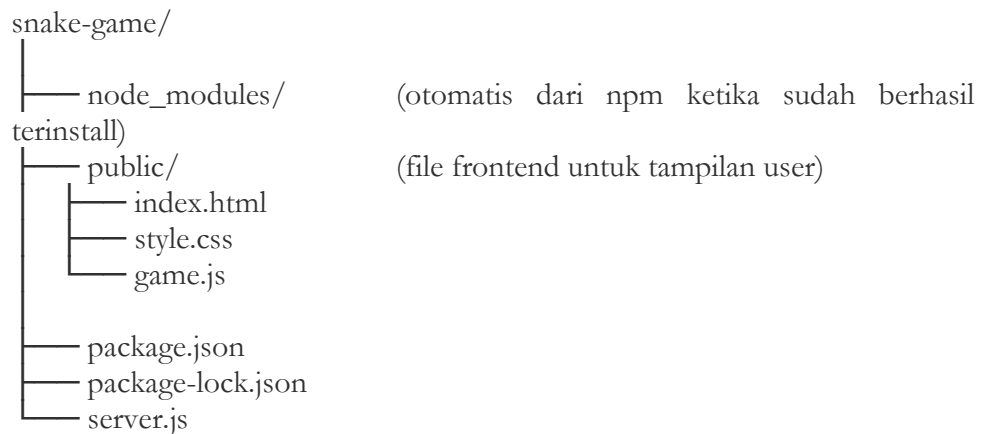
Digunakan untuk memindahkan posisi kepala ular ke koordinat baru berdasarkan *input* arah.

2. **Scaling**

Digunakan untuk mengatur ukuran objek agar tetap proporsional terhadap *viewport*.

4. **Deteksi Tabrakan (*Collision Detection*):** Menerapkan logika matematis untuk mendeteksi perpotongan koordinat antara kepala ular dengan makanan, dinding (*clipping window*), atau tubuhnya sendiri.

5. **Struktur Folder Proyek Snake Game**



6. **Langkah Pengerjaan Proyek**

- 1) Buatlah folder dengan nama snake-game.

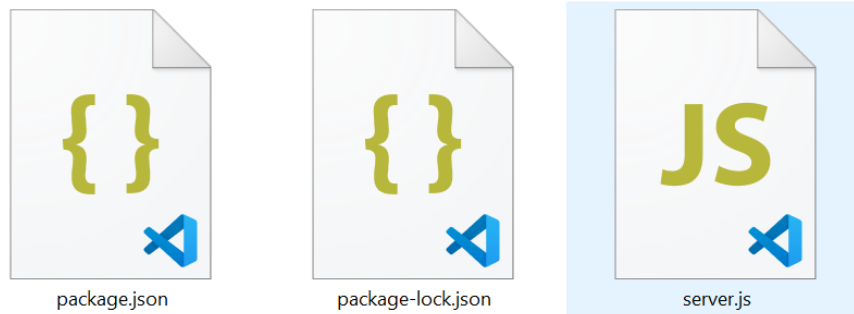


snake-game

- 2) Buatlah file berikut pada folder snake-game yaitu:

- package.json,
- package-lock.json, dan

- server.js



- 3) Buka foldered snake-game dan buatlah sebuah folder baru dengan nama public.
- 4) Buka folder public dan buatlah tiga file yaitu:
 - index.html,
 - style.css, dan
 - game.js



- 5) Tambahkan script berikut pada file **server.js**, file ini digunakan untuk menjalankan server dengan **Express.js**. Fungsi file ini untuk menyajikan file dari folder public dan menjalankan aplikasi di browser.

server.js

```
const express = require("express");
const app = express();

app.use(express.static("public"));

app.listen(3000, () => {
  console.log("Server running di http://localhost:3000");
});
```

- 6) Tambahkan script berikut pada file **package.json**, file ini untuk konfigurasi proyek Node.js,. Fungsi file ini untuk menyimpan daftar library dan menjalankan project (npm start).
- 7) Pada folder **public** tambahkan kode berikut.

a. Tambahkan kode berikut pada file **index.html**

```
<!DOCTYPE html>
<html>
<head>
  <title>Snake Game</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<h2>Snake Game</h2>
<canvas id="game" width="400" height="400"></canvas>

<script src="game.js"></script>
</body>
</html>
```

b. Tambahkan kode berikut pada file **style.css**

```
body {
  text-align: center;
  font-family: Arial;
}

canvas {
  border: 2px solid black;
}
```

c. Tambahkan kode berikut pada file **game.js**. Fungsi kode ini adalah untuk mengatur grafika (Canvas), logika game (gerak, objek, dll).

- 1) Buatlah sebuah fungsi untuk menggerakkan ular bergerak dari kanan ke kiri, kiri ke kanan, atas ke bawah, bawah ke atas, menambahkan kepala ular dna menghapus ekor ular.

```
//Fungsi moveSnake New pada level ini dapat menabrak dinding tembus
// GERAK SNAKE + WRAP DINDING

function moveSnake() {
  let head = {
    x: snake[0].x + dx,
    y: snake[0].y + dy
  };

  // ---- WRAP KANAN → KIRI ----
  if (head.x >= canvas.width) {
    head.x = 0;
  }

  // ---- WRAP KIRI → KANAN ----
  if (head.x < 0) {
    head.x = canvas.width - 20;
  }

  // ---- WRAP BAWAH → ATAS ----
  if (head.y >= canvas.height) {
    head.y = 0;
  }

  // ---- WRAP ATAS → BAWAH ----
  if (head.y < 0) {
    head.y = canvas.height - 20;
  }

  // Tambahkan kepala baru
  snake.unshift(head);

  // Hapus ekor
  snake.pop();
}
```

- 2) Buatlah fungsi checkFood untuk mengecek makanan, menambah panjang ular, dan menambah ekor ular.

```
// FUNGSI CEK MAKANAN

function checkFood() {
  // Jika kepala snake sama dengan posisi food
  if (snake[0].x === food.x && snake[0].y === food.y) {

    // Tambahkan panjang snake
    snake.push({});

    // Generate food baru
    food = generateFood();

    // Tambah skor
    score += 1;
  }
}
```

- 3) Buatlah fungsi checkCollision atau tabrakan dengan diri sendiri maka game akan berhenti.

```
// FUNGSI CEK TABRAKAN

function checkCollision() {
  const head = snake[0];

  // ---- TABRAK DIRI SENDIRI ----
  for (let i = 1; i < snake.length; i++) {
    if (head.x === snake[i].x && head.y === snake[i].y) {
      gameOver = true;
    }
  }
}
```

- 4) Buatlah fungsi drawGameOver untuk untuk menandakan permainan telah berakhir.

```
// FUNGSI GAME OVER

function drawGameOver() {
  ctx.fillStyle = "black";
  ctx.font = "30px Arial";
  ctx.fillText("GAME OVER", 110, 200);

  ctx.font = "16px Arial";
  ctx.fillText("Score: " + score, 150, 230);
}
```

- 5) Buatlah sebuah kontrol untuk memainkan game menggunakan keyboard yaitu tombol arah ke atas, ke bawah, ke kanan dan ke kiri.

```
// KONTROL KEYBOARD KETIMA MEMAINKAN GAME

document.addEventListener("keydown", function (event) {
  const key = event.key;

  // Atas
  if (key === "ArrowUp" && dy === 0) {
    dx = 0;
    dy = -20;
  }

  // Bawah
  else if (key === "ArrowDown" && dy === 0) {
    dx = 0;
    dy = 20;
  }

  // Kiri
  else if (key === "ArrowLeft" && dx === 0) {
    dx = -20;
    dy = 0;
  }

  // Kanan
  else if (key === "ArrowRight" && dx === 0) {
    dx = 20;
    dy = 0;
  }

  // Restart game jika game over
  if (gameOver && key === "Enter") {
    location.reload();
  }
});
```

- 6) Buatlah sebuah kontrol untuk memainkan game menggunakan mouse dengan menekan kursor pada arah ke atas, ke bawah, ke kanan dan ke kiri.

```
// KONTROL MOUSE (OPSIONAL)

canvas.addEventListener("click", function (event) {
  // Ambil posisi mouse
  const rect = canvas.getBoundingClientRect();
  const mouseX = event.clientX - rect.left;
  const mouseY = event.clientY - rect.top;

  const head = snake[0];

  // Hitung selisih posisi
  const diffX = mouseX - head.x;
  const diffY = mouseY - head.y;

  // Tentukan arah dominan
  if (Math.abs(diffX) > Math.abs(diffY)) {
    dx = diffX > 0 ? 20 : -20;
    dy = 0;
  } else {
    dx = 0;
    dy = diffY > 0 ? 20 : -20;
  }
});
```

- 7) Buatlah sebuah fungsi gameLoop untuk mengulang game jika permainan selesai.

```
// GAME LOOP (INTI GAME)

function gameLoop() {
  // Bersihkan canvas setiap frame
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  // Jika game over → tampilkan teks dan berhenti
  if (gameOver) {
    drawGameOver();
    return;
  }

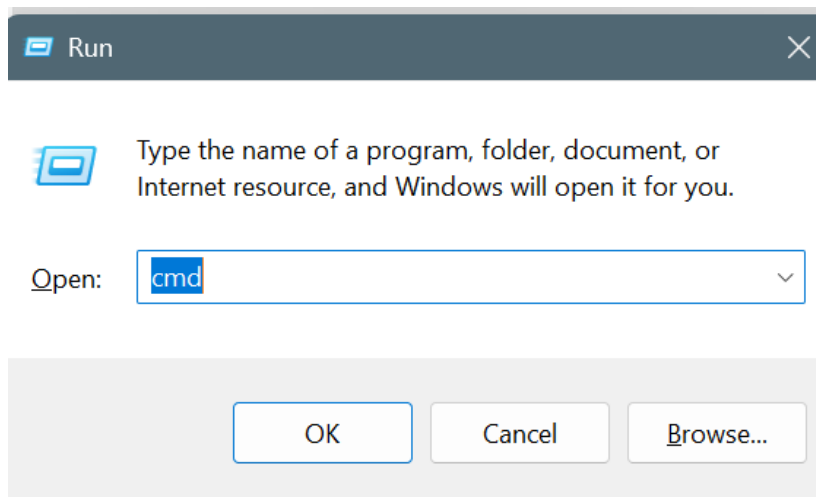
  // Jalankan logika game
  moveSnake();
  checkFood();
  checkCollision();

  // Gambar semua objek
  drawSnake();
  drawFood();
  drawScore();

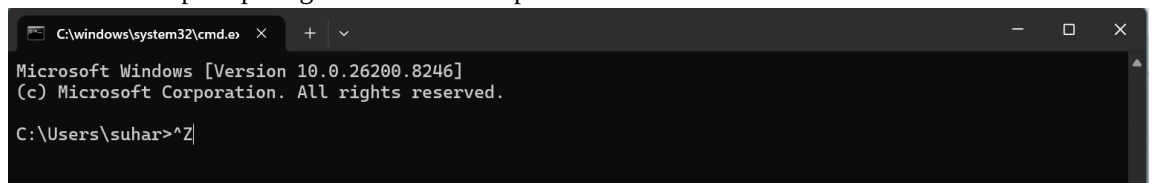
  // Ulangi loop (animasi)
  setTimeout(gameLoop, 120);
}

// Mulai game
gameLoop();
```

- 8) Cara menjalankan snake-game
9) Buka teminal atau CMD, kemudian tekan enter atau klok OK.

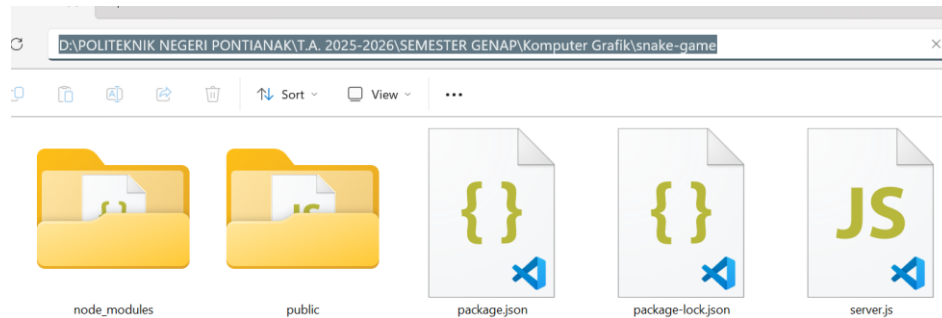


- 10) Maka akan tampil seperti gambar terminal seperti berikut.



- 11) Selanjutnya, pindah ke direktori penyimpanan snake-game.

- 12) Buka folder tempat penyimpanan snake-game, kemudian klik pada bagian direktori seperti gambar berikut untuk menyalin alamat direktori snake-game.



- 13) Maka akan tersalin alamat direktori:

D:\ cd POLITEKNIK NEGERI PONTIANAK\T.A. 2025-2026\SEMESTER GENAP\Komputer Grafik\snake-game

atau (jika tidak terbaca ubah slash dari \ menjadi /)

D:\>cd POLITEKNIK NEGERI PONTIANAK/T.A. 2025-2026/SEMESTER GENAP/Komputer Grafik/snake-game

- 14) Selanjutnya akan pindah ke direktori snake-game

```
C:\windows\system32\cmd.exe x + v
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\suhar>D:

D:\>POLITEKNIK NEGERI PONTIANAK\T.A. 2025-2026\SEMESTER GENAP\Komputer Grafik\snake-game
'POLITEKNIK' is not recognized as an internal or external command,
operable program or batch file.

D:\>POLITEKNIK NEGERI PONTIANAK/T.A. 2025-2026/SEMESTER GENAP/Komputer Grafik/snake-game
'POLITEKNIK' is not recognized as an internal or external command,
operable program or batch file.

D:\>POLITEKNIK NEGERI PONTIANAK\T.A. 2025-2026\SEMESTER GENAP\Komputer Grafik\snake-game
'POLITEKNIK' is not recognized as an internal or external command,
operable program or batch file.

D:\>cd POLITEKNIK NEGERI PONTIANAK\T.A. 2025-2026\SEMESTER GENAP\Komputer Grafik\snake-game
D:\POLITEKNIK NEGERI PONTIANAK\T.A. 2025-2026\SEMESTER GENAP\Komputer Grafik\snake-game>
```

- 15) Ketikkan npm install, kemudian tekan enter. Maka akan tampil bahwa npm sudah berhasil terinstall.

```
D:\POLITEKNIK NEGERI PONTIANAK\T.A. 2025-2026\SEMESTER GENAP\Komputer Grafik\snake-game>npm install

up to date, audited 69 packages in 842ms

15 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

D:\POLITEKNIK NEGERI PONTIANAK\T.A. 2025-2026\SEMESTER GENAP\Komputer Grafik\snake-game>
```

- 16) Ketikkan npm start, kemudian tekan enter. Maka akan tampil pesan bahwa Server running pada http://localhost:3000.

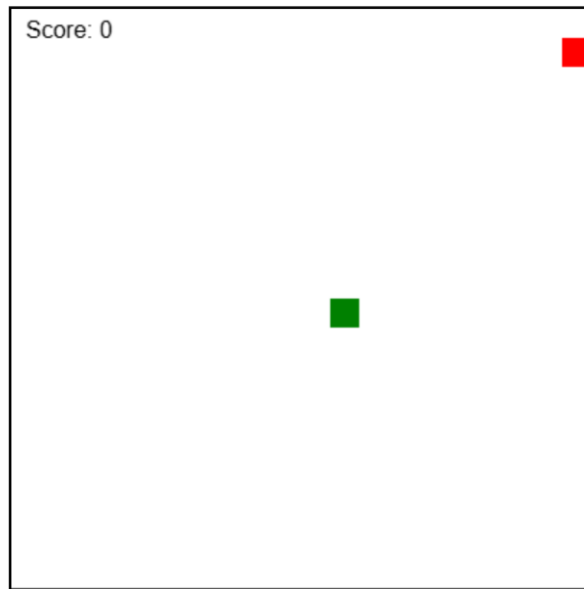
```
D:\POLITEKNIK NEGERI PONTIANAK\T.A. 2025-2026\SEMESTER GENAP\Komputer Grafik\snake-game>npm start

> snake-game@1.0.0 start
> node server.js

Server running di http://localhost:3000
```

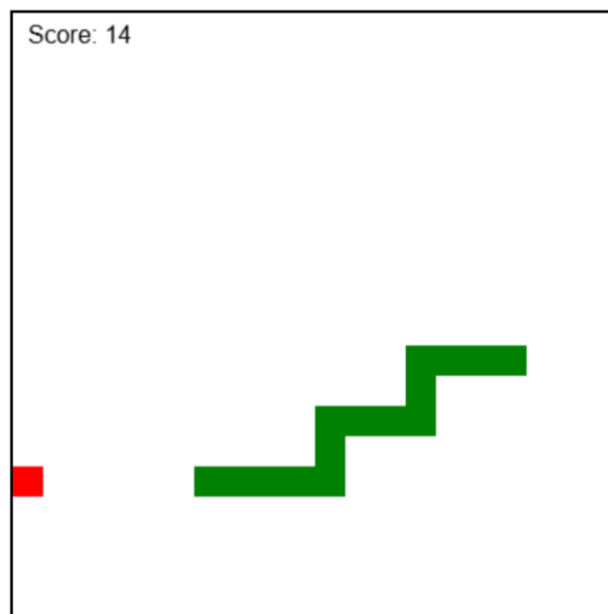
- 17) Selanjutnya ketikkan di browser : <http://localhost:3000/> maka akan tampil tampilan Snake Game.

Snake Game



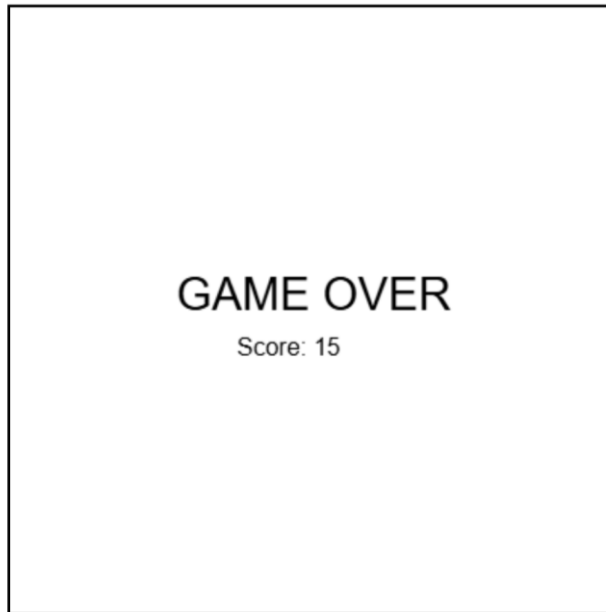
- 18) Game sudah dapat di mainkan.

Snake Game



19) Tampilan ketika game over.

Snake Game



20) Latihan PjBL

Terapkan konsep berikut pada game yang telah dibuat sebelumnya.

1. Manajemen *level* yaitu *level easy*, *level medium*, dan *level hard*.
2. Pemain akan naik *level* jika telah mencapai skor pada masing-masing *level*. *Level easy* skor 25, *level medium* skor 50, dan *level hard* skor lebih dari 50.
3. *Pause system* pada *game* setelah menyelesaikan *level* sebelumnya dan akan melanjutkan *level* berikutnya.
4. Tombol antarmuka pemain yaitu tombol "Lanjut Level".
5. *Dynamic speed* pada *game* akan meningkat sesuai dengan levelnya. *Level easy* kecepatan *game* 200, *level medium* kecepatan *game* 150, dan *level hard* kecepatan *game* 75.
6. *Update file* index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Snake Game Level</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<h1>Snake Game</h1>

<canvas id="game" width="400" height="400"></canvas>

<!-- PANEL LEVEL -->
<div id="levelPanel" class="hidden">
  <h2 id="levelText"></h2>
  <p id="scoreText"></p>

  <button id="nextLevelBtn">
    Mulai Level Selanjutnya
  </button>
</div>

<script src="game.js"></script>

</body>
</html>
```

7. Update file game.js

```
// =====
// CANVAS
// =====
const canvas = document.getElementById("game");
const ctx = canvas.getContext("2d");

// =====
// PANEL LEVEL
// =====
const levelPanel = document.getElementById("levelPanel");
const levelText = document.getElementById("levelText");
const scoreText = document.getElementById("scoreText");
const nextLevelBtn = document.getElementById("nextLevelBtn");

// =====
// SNAKE
// =====
let snake = [{x:200,y:200}];

let dx = 20;
let dy = 0;

// =====
// FOOD
// =====
let food = generateFood();

// =====
// SCORE
// =====
let score = 0;
```

```
// =====
// LEVEL
// =====
let level = "Easy";

// speed game
let gameSpeed = 150;

// pause level
let pauseLevel = false;

// game over
let gameOver = false;

// =====
// GENERATE FOOD
// =====
function generateFood(){
  return{
    x:Math.floor(Math.random()*20)*20,
    y:Math.floor(Math.random()*20)*20
  };
}

// =====
// DRAW SNAKE
// =====
function drawSnake(){
  snake.forEach(part=>{
    ctx.fillStyle="green";
    ctx.fillRect(part.x,part.y,20,20);
  });
}

// =====
// DRAW FOOD
// =====
function drawFood(){
  ctx.fillStyle="red";
  ctx.fillRect(food.x,food.y,20,20);
}

// =====
// DRAW SCORE
// =====
function drawScore(){
  ctx.fillStyle="black";
  ctx.font="16px Arial";

  ctx.fillText("Score : "+score,10,20);
  ctx.fillText("Level : "+level,300,20);
}
```

```

// =====
// MOVE SNAKE
// =====
function moveSnake(){

  let head = {
    x:snake[0].x + dx,
    y:snake[0].y + dy
  };

  // wrap easy-medium
  if(level !== "Hard"){

    if(head.x >= canvas.width)
      head.x = 0;

    if(head.x < 0)
      head.x = canvas.width - 20;

    if(head.y >= canvas.height)
      head.y = 0;

    if(head.y < 0)
      head.y = canvas.height - 20;
  }

  // hard mode
  else{
    if(
      head.x < 0 ||
      head.x >= canvas.width ||
      head.y < 0 ||
      head.y >= canvas.height
    ){
      gameOver = true;
    }
  }

  snake.unshift(head);
  snake.pop();
}

// =====
// CHECK FOOD
// =====
function checkFood(){

  if(
    snake[0].x === food.x &&
    snake[0].y === food.y
  ){

```

```
// =====  
// UPDATE LEVEL  
// =====  
function updateLevel(){  
  
    // EASY  
    if(score <= 25){  
  
        if(level !== "Easy"){  
            showLevelTransition("Easy");  
        }  
  
        level = "Easy";  
        gameSpeed = 200;  
    }  
  
    // MEDIUM  
    else if(score <= 50){  
  
        if(level !== "Medium"){  
            showLevelTransition("Medium");  
        }  
  
        level = "Medium";  
        gameSpeed = 150;  
    }  
  
    // HARD  
    else{  
  
        if(level !== "Hard"){  
            showLevelTransition("Hard");  
        }  
  
        level = "Hard";  
        gameSpeed = 75;  
    }  
}  
  
// =====  
// LEVEL TRANSITION  
// =====  
function showLevelTransition(newLevel){  
  
    pauseLevel = true;  
  
    levelPanel.classList.remove("hidden");  
  
    levelText.innerText =  
        "LEVEL " + newLevel;  
  
    scoreText.innerText =
```

```
// =====
// NEXT LEVEL BUTTON
// =====
nextLevelBtn.addEventListener("click",()=>{

    pauseLevel = false;

    levelPanel.classList.add("hidden");

    gameLoop();
});

// =====
// COLLISION BODY
// =====
function checkCollision(){

    const head = snake[0];

    for(let i=1;i<snake.length;i++){

        if(
            head.x === snake[i].x &&
            head.y === snake[i].y
        ){
            gameOver = true;
        }
    }
}

// =====
// GAME OVER
// =====
function drawGameOver(){

    ctx.fillStyle="black";

    ctx.font="30px Arial";

    ctx.fillText(
        "GAME OVER",
        100,
        200
    );
}

// =====
// KEYBOARD
// =====
document.addEventListener("keydown", (event)=>{

    const key = event.key;
```

```

    dx=0;
    dy=-20;
  }

  else if(key==="ArrowDown" && dy===0){
    dx=0;
    dy=20;
  }

  else if(key==="ArrowLeft" && dx===0){
    dx=-20;
    dy=0;
  }

  else if(key==="ArrowRight" && dx===0){
    dx=20;
    dy=0;
  }
});

// =====
// GAME LOOP
// =====
function gameLoop(){
  if(pauseLevel) return;

  ctx.clearRect(
    0,
    0,
    canvas.width,
    canvas.height
  );

  if(gameOver){
    drawGameOver();
    return;
  }

  moveSnake();
  checkFood();
  checkCollision();
  drawSnake();
  drawFood();
  drawScore();

  setTimeout(gameLoop,gameSpeed);
}

// =====
// START GAME
// =====
gameLoop();

```

8. Update file style.css

```
body{
  text-align:center;
  font-family:Arial;
}

canvas{
  border:2px solid black;
  background:#eee;
}

#levelPanel{
  margin-top:20px;
}

.hidden{
  display:none;
}

button{
  padding:10px 20px;
  font-size:16px;
  cursor:pointer;
}
```

21) Tugas PjBL

1. Modifikasi *game* dengan level menjadi 5 level.
2. Skor setiap level yaitu level 1 skor 25, level 2 skor 50, level 3 skor 75, level 4 skor 100, level 5 skor lebih dari 100.
3. Kecepatan level meningkat sesuai dengan tingkat levelnya. Level 1 *speed* 250, level 2 *speed* 200, level 3 *speed* 150, level 4 *speed* 100, level 5 *speed* 50.

4. Tahap Pengujian (*Testing*)

Tahap pengujian (testing) dilakukan untuk mengevaluasi kualitas game sebelum di relase kepada pengguna. Pada PjBL ini pengujian dilakukan dengan menggunakan tiga metode yaitu: *black-box testing*, *visual testing*, dan *functional testing*. Target pengujian harus menunjukkan bahwa game dapat berjalan sesuai rancangan dengan tingkat keberhasilan fitur $\geq 75\%$.

a. Black-box Testing

Black-box Testing adalah metode pengujian perangkat lunak yang berfokus pada input dan output tanpa melihat kode program di dalam sistem. Pada pengujian ini:

- 1) Penguji hanya mencoba fitur game,
- 2) Memberikan input,
- 3) Lalu melihat apakah hasil sesuai harapan

Tujuan Pengujian ini adalah untuk:

- 1) Memastikan game berjalan sesuai kebutuhan pengguna.
- 2) Menguji fungsi utama game dari sisi pemain

Fokus Pengujian

- 1) Kontrol keyboard,
- 2) Skor,
- 3) Collision,
- 4) Level,
- 5) Game over.

Skenario Black Box Testing

No	Fitur yang Diuji	Input	Output yang Diharapkan	Output Hasil Pengujian
1	Gerak ular ke kanan	Tombol →	Ular bergerak ke kanan	
2	Gerak ular ke kiri	Tombol ←	Ular bergerak ke kiri	
3	Gerak ular ke atas	Tombol ↑	Ular bergerak ke atas	
4	Gerak ular ke bawah	Tombol ↓	Ular bergerak ke bawah	
5	Makan makanan	Ular menyentuh food	Skor bertambah +1	
6	Pergantian	Skor > 50	Level berubah	

No	Fitur yang Diuji	Input	Output yang Diharapkan	Output Hasil Pengujian
	level		menjadi Medium	
7	Pergantian level	Skor > 100	Level berubah menjadi Hard	
8	Tabrak badan sendiri	Snake menyentuh tubuh	Game Over tampil	
9	Tombol lanjut level	Klik tombol	Game kembali berjalan	
10	Restart game	Refresh / Enter	Game kembali ke awal	

Hasil Pengujian

Berdasarkan hasil pengujian, seluruh fitur utama game Snake berhasil dijalankan sesuai dengan skenario yang telah dirancang. Input keyboard berupa tombol arah dapat menggerakkan ular dengan benar ke setiap arah. Sistem skor juga berhasil bertambah ketika ular memakan makanan. Fitur perubahan level otomatis berdasarkan skor berjalan sesuai logika yang telah ditentukan, yaitu level Easy, Medium, dan Hard. Selain itu, sistem collision detection berhasil menampilkan kondisi Game Over ketika ular menabrak tubuhnya sendiri. Secara keseluruhan, hasil pengujian menunjukkan bahwa game telah mampu merespons input pengguna dengan baik dan menghasilkan output sesuai harapan.

b. Visual Testing

Visual Testing adalah metode pengujian yang digunakan untuk mengevaluasi tampilan visual, animasi, dan kualitas rendering grafika pada aplikasi atau game.

Tujuan Pengujian

- 1) Memastikan objek tampil dengan benar.
- 2) Memastikan tidak ada distorsi visual.
- 3) Memastikan animasi berjalan halus.

Fokus Pengujian

- 1) Tampilan snake,
- 2) Warna objek,
- 3) Posisi food,
- 4) Animasi gerak,
- 5) Teks skor,
- 6) Viewport.

Skenario Visual Testing

No	Komponen Visual	Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Snake	Gerakan ular	Gerakan halus tanpa flicker	
2	Food	Posisi makanan	Food tampil penuh di canvas	
3	Skor	Tampilan teks	Teks score terbaca jelas	
4	Level	Pergantian level	Informasi level tampil benar	
5	Collision	Snake tabrak badan	Tampilan Game Over muncul	
6	Rendering	Objek grafis	Tidak ada distorsi objek	
7	Grid posisi	Snake bergerak	Snake tetap berada pada grid	
8	Warna objek	Snake & food	Warna tampil konsisten	
9	Canvas	Area permainan	Objek tidak keluar viewport	
10	Animasi	Pergerakan	Tidak terjadi lag berat	

Hasil Pengujian

Hasil pengujian visual menunjukkan bahwa seluruh objek grafis pada game Snake dapat ditampilkan dengan baik pada area canvas. Gerakan ular terlihat stabil dan tidak mengalami flickering maupun distorsi visual yang mengganggu permainan. Posisi makanan berhasil muncul secara penuh di dalam viewport sehingga mudah dikenali pemain. Tampilan teks skor dan level juga dapat dibaca dengan jelas selama permainan berlangsung. Selain itu, animasi perpindahan ular antar grid berjalan cukup halus dan konsisten. Tidak ditemukan objek keluar dari area permainan maupun kerusakan tampilan grafis selama pengujian dilakukan. Dengan demikian, kualitas visual game dinilai telah memenuhi kebutuhan dasar game 2D sederhana berbasis grafika komputer..

c. Functional Testing

Functional Testing adalah metode pengujian yang digunakan untuk memastikan setiap fungsi pada game berjalan sesuai rancangan sistem.

Tujuan Pengujian

- 1) Memastikan seluruh fitur bekerja.
- 2) Memastikan logika program benar.
- 3) Menguji integrasi antar fitur.

Focus Pengujian

- 1) collision detection,
- 2) scoring,
- 3) level system,
- 4) pause system,
- 5) keyboard event.

Skenario Functional Testing

No	Fungsi	Proses Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Sistem skor	Snake makan food	Score bertambah	
2	Collision badan	Snake tabrak tubuh	Game Over aktif	
3	Collision dinding hard mode	Snake tabrak dinding	Game Over	
4	Wrap mode easy	Snake keluar kanan	Muncul di kiri	
5	Sistem level	Score mencapai batas	Level berubah otomatis	
6	Pause level	Naik level	Game berhenti sementara	
7	Tombol lanjut	Klik tombol	Game lanjut	
8	Spawn food	Food dimakan	Food baru muncul random	
9	Kecepatan level	Naik level	Speed meningkat	
10	Input keyboard	Tombol arah	Snake berubah arah	

Hasil Pengujian

Pengujian fungsi sistem menunjukkan bahwa seluruh logika utama permainan dapat berjalan sesuai rancangan. Sistem skor berhasil bertambah ketika ular memakan makanan, dan makanan baru muncul secara acak pada area permainan. Mekanisme collision detection juga bekerja dengan baik, baik pada tabrakan tubuh sendiri maupun tabrakan dinding pada mode Hard. Selain itu, sistem wrap pada level Easy dan Medium berhasil memindahkan ular ke sisi berlawanan

canvas ketika melewati batas area permainan. Fitur perubahan level otomatis berdasarkan skor berjalan sesuai aturan yang telah ditentukan. Kecepatan permainan meningkat secara bertahap pada setiap level sehingga memberikan peningkatan tingkat kesulitan permainan. Fungsi pause level dan tombol lanjut level juga berhasil menghentikan permainan sementara dan melanjutkan kembali tanpa error. Secara keseluruhan, hasil pengujian menunjukkan bahwa integrasi antar fungsi game berjalan stabil dan sesuai desain sistem.

5. Tahap Rilis (*Release*)

Tahap rilis merupakan tahap akhir dalam proses pengembangan game pada metode Game Development Life Cycle (GDLC). Pada tahap ini, game yang telah selesai dikembangkan dan lolos proses pengujian dan dapat dipublikasikan agar untuk digunakan oleh pengguna.

Dokumentasi

Dokumentasi dari proyek hasil proyek ini antara lain sebagai berikut:

- 1) *screenshot game*,
- 2) *video gameplay*,
- 3) *source code*, dan
- 4) laporan pengembangan.

Platform

Dalam PjBL Game Snake, tahap rilis dilakukan dengan menempatkan game pada *platform* berbasis *web* (versi tidak berbayar atau versi ujicoba) sehingga dapat diakses melalui browser tanpa perlu instalasi tambahan. Platform yang dapat digunakan antara lain sebagai berikut.

- 1) Vercel : <https://vercel.com/>
- 2) Netlify : <https://www.netlify.com/>
- 3) Render : <https://render.com/>
- 4) GitHub Pages : <https://docs.github.com/en/pages>

Distribusi

Distribusi hasil dari proyek yang telah dibuat dibagikan kepada pengguna melalui:

- 1) Link *game* salah satu dari *platform* diatas.
- 2) *Repository project* yang berisi file proyek pada link GutHub *Pages*.
- 3) Maupun video demonstrasi proyek game melalui *channel* Youtube.

Tujuan Tahap Rilis

Tujuan pada tahap rilis adalah sebagai berikut.

- 1) menyebarkan hasil pengembangan game kepada pengguna,
- 2) memastikan game dapat dijalankan pada lingkungan nyata,
- 3) memperoleh umpan balik dari pengguna,
- 4) mendokumentasikan hasil implementasi proyek,
- 5) dan mengevaluasi kesiapan game untuk digunakan secara umum.